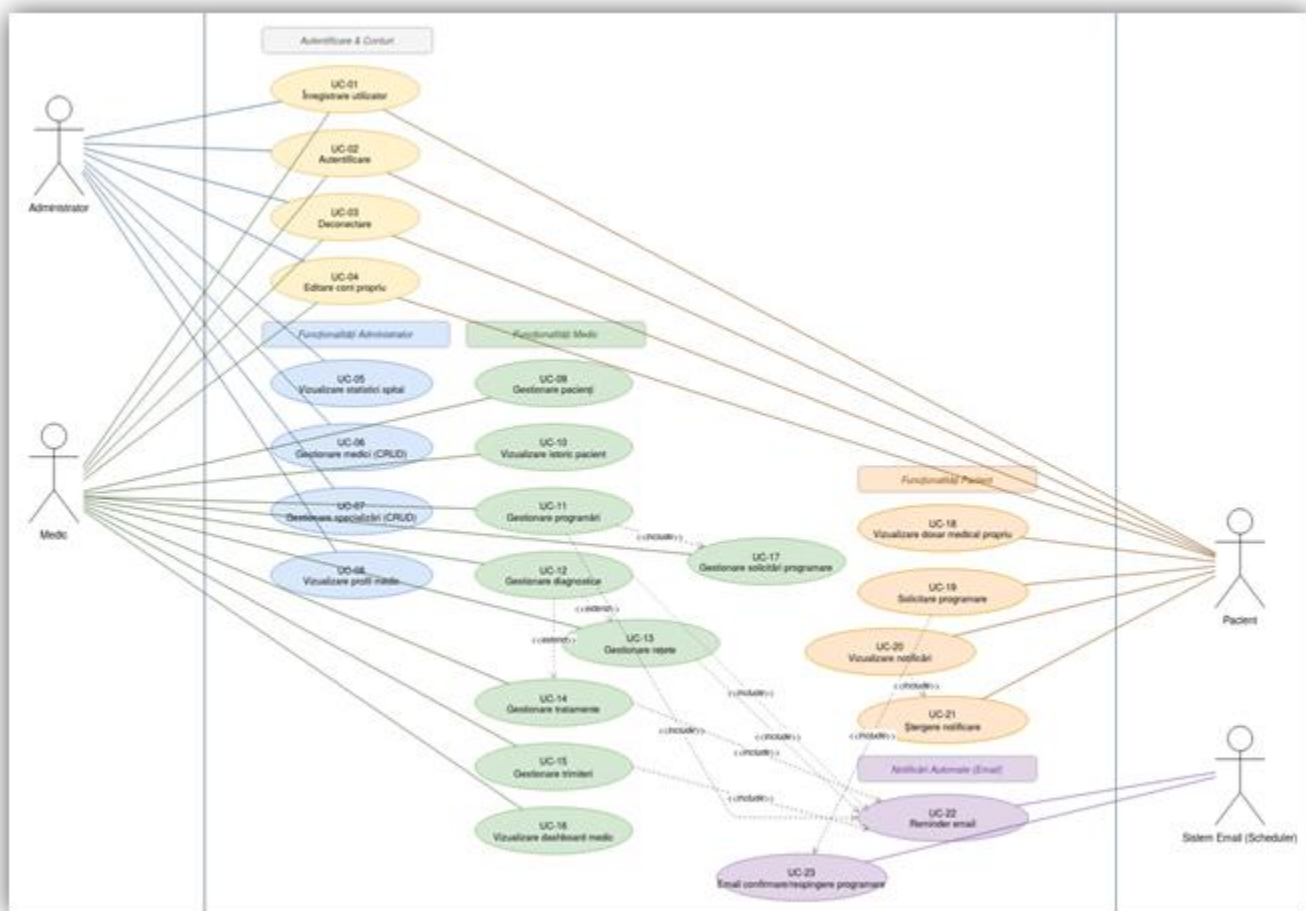


6. E-SPITAL – APLICAȚIE DE GESTIUNE A ISTORICULUI MEDICAL

Prezentare Generală

Aplicația dezvoltată în cadrul acestei lucrări propune o soluție clară: un sistem info-tehnologic bazat pe arhitectură client-server, unde datele pacienților sunt stocate într-o bază de date securizată, iar medicii, administrația, precum și pacienții interacționează printr-o interfață web. În acest fel, aplicația dată integrează funcții de creare și actualizare a profilului pacientului, programări medicale, managementul rețetelor, diagnosticelor, tratamentelor și trimiterilor, precum și autentificare cu roluri diferite (medic, pacient,

administrator).



Figură 9 – Diagrama de utilizare al aplicației E-Spital

Fluxul aplicației poate fi prezentat în diagrama de utilizare de la [figura 9](#).

Actorii Sistemului

Aplicația E-Spital identifică trei tipuri de actori principali (utilizatori) care interacționează cu sistemul:

Actor	Tip	Descriere
Administrator (Admin)	Actor primar	Gestionează medicii și specializările. Vizualizează statistici generale ale

		spitalului. Nu are acces la datele medicale ale pacienților.
Medic	Actor primar	Gestionează pacienții și dosarele lor medicale (programări, diagnostice, rețete, tratamente, trimiteri). Are o specializare asociată.
Pacient	Actor secundar	Vizualizează propriul istoric medical și poate solicita programări la medici. Primește notificări și remindere prin email.
Sistem de Email (Scheduler)	Actor secundar	Trimite notificări automate zilnic la ora 08:00 pentru remindere programări (cu o zi înainte), expirare rețete, tratamente, trimiteri. Trimite notificări de confirmare pentru o programare acceptată sau respinsă.

Cazuri de Utilizare

Cazurile de utilizare surprind acțiunile pe care actorii le săvârșesc. Acestea sunt reprezentate în [figura 9](#), dar și în tabelul de mai jos, mai detaliat:

ID	Caz de Utilizare	Actor	Descriere Scurtă
UC-01	Înregistrare utilizator	Orice utilizator	Crearea unui cont nou cu rol (admin/medic/pacient) și validarea datelor.
UC-02	Autentificare	Admin, Medic, Pacient	Logare cu email și parolă, redirectionare la dashboard specific.
UC-03	Deconectare	Admin, Medic, Pacient	Invalidarea sesiunii și redirectionarea la pagina de login.
UC-04	Editare cont propriu	Admin, Medic, Pacient	Vizualizare și modificare date personale; ștergere cont.

UC-05	Vizualizare statistici spital	Admin	Dashboard cu statistici agregate pe perioade de timp.
UC-06	Gestionare medici	Admin	CRUD complet pentru medici (adăugare, editare, ștergere, vizualizare profil).
UC-07	Gestionare specializări	Admin	CRUD complet pentru specializări medicale.
UC-08	Vizualizare profil medic	Admin	Afișare date personale și activitate completă a unui medic.
UC-09	Gestionare pacienți	Medic	Adăugare pacient nou sau asociere pacient existent; editare și ștergere.
UC-10	Vizualizare istoric pacient	Medic	Afișare completă a dosarului medical al unui pacient.
UC-11	Gestionare programări	Medic	Creare, editare (rapidă sau completă), ștergere programări; validare status/dată.
UC-12	Gestionare diagnostice	Medic	CRUD diagnostice asociate programărilor.
UC-13	Gestionare rețete	Medic	CRUD rețete asociate diagnosticelor.
UC-14	Gestionare tratamente	Medic	CRUD tratamente recomandate.
UC-15	Gestionare trimiteri	Medic	CRUD trimiteri medicale.
UC-16	Vizualizare dashboard medic	Medic	Afișare solicitări, programări curente, statistici săptămânale, expirări.
UC-17	Gestionare solicitări programare	Medic	Acceptare sau respingere solicitări de programare de la pacienți.
UC-18	Vizualizare dosar medical propriu	Pacient	Pacientul vizualizează propriul istoric medical.
UC-19	Solicitare programare	Pacient	Trimiterea unei cereri de programare la un medic cu interval de date preferat.

UC-20	Vizualizare notificări	Pacient	Afișarea răspunsurilor primite la solicitările de programare.
UC-21	Ștergere notificare	Pacient	Pacientul șterge o notificare din lista sa.
UC-22	Primire reminder email programare	Pacient (automat)	Email automat trimis cu o zi înainte de programare.
UC-23	Primire email confirmare/respingere	Pacient (automat)	Email automat la acceptarea sau respingerea unei solicitări de programare.

Scenarii de Utilizare

Un scenariu reprezintă descrierea unui exemplu concret de folosire a sistemului. Avem ca și exemple următoarele scenarii:

UC-11: Gestionare Programări

Câmp	Detalii
Identificator	UC-11
Denumire	Gestionare Programări
Scop	Crearea, vizualizarea, editarea și ștergerea unei programări
Actor principal	Medic
Declanșator	Medicul selectează programări
Precondiții	Medicul este autentificat; există pacienți asociați medicului.
Flux principal	1. Medicul navighează la pagina 'Programări'. 2. Vizualizează calendarul și listele de programări. 3. Creează o programare nouă: selectează pacientul, introduce data și detaliile. 4. Modifică rapid statusul (finalizat/amânat/ștergere) prin butoanele din tabel. 5. Sau editează complet programarea (deschide formular de editare).
Flux alternativ 1	Finalizare programare viitoare: sistemul returnează eroare 'Nu poți finaliza o programare care nu a avut loc încă.'

Flux alternativ 2	Marcare ca 'viitoare' a unei programări din trecut: eroare 'O programare marcată ca viitoare trebuie să fie în viitor.'
Postcondiții	Programarea este creată/actualizată/ștearsă în baza de date.
Reguli business	Status acceptat: viitoare finalizată amânată. Finalizarea necesită data în trecut. Statusul 'viitoare' necesită data în viitor.

UC-19: Solicitare Programare (Pacient)

Câmp	Detalii
Identificator	UC-19
Denumire	Solicitare Programare
Scop	Trimiterea unei solicitări de programare către un medic, și eventual, crearea unei programări, dacă este acceptată
Actor principal	Pacient
Declanșator	Pacientul selectează opțiunea 'Solicită o programare'
Precondiții	Pacientul este autentificat; există medici disponibili în sistem.
Flux principal	1. Pacientul accesează formularul de solicitare programare. 2. Selectează medicul dorit din lista disponibilă. 3. Introduce data de start și data de final a intervalului preferat. 4. Adaugă opțional un mesaj pentru medic. 5. Trimite solicitarea. 6. Solicitarea apare pe dashboard-ul medicului cu statusul 'trimisă'. 7. Medicul acceptă → programarea se creează; pacientul primește notificare + email. 8. Sau medicul respinge → solicitarea capătă statusul 'respinsă'; pacientul primește notificare + email.
Flux alternativ	Data de start nu este ulterioară zilei de azi: validare 'after:today'.
Postcondiții	Solicitarea este înregistrată în tabela SolicitareProgramare; pacientul primește feedback.
Reguli business	Statusuri posibile: trimisă rezolvată respinsă. Data de start trebuie să fie în viitor. Data de end trebuie să fie după data de start.

Cerințe Funcționale

Cerințele funcționale descriu serviciile pe care sistemul le oferă. Mai departe avem următoarele cerințe:

Autentificare și Gestionare Conturi

Accesul utilizatorilor în sistem trebuie să fie reglementat, fiecare actor operând exclusiv în perimetrul de drepturi corespunzător rolului său.

- CF-01: Sistemul permite înregistrarea utilizatorilor cu roluri distincte: admin, medic, pacient.
- CF-02: Sistemul validează datele de înregistrare (client-side JS + server-side Laravel).
- CF-03: Parolele sunt stocate criptat (bcrypt hash).
- CF-04: Utilizatorul se poate autentifica cu email și parolă.
- CF-05: După autentificare, utilizatorul este redirecționat automat spre dashboard-ul rolului său.
- CF-06: Utilizatorul se poate deconecta, cu invalidarea sesiunii și regenerarea token-ului CSRF.
- CF-07: Utilizatorul autentificat poate vizualiza și edita datele propriului cont.
- CF-08: Utilizatorul autentificat poate șterge propriul cont.
- CF-09: Dacă rolul este medic, trebuie selectată o specializare validă la înregistrare.
- CF-10: Dacă rolul este pacient, trebuie specificat statutul de asigurat.

Funcționalități Administrator

Un administrator poate avea următoarele funcții:

- CF-11: Administratorul vizualizează statistici agregate (pacienți, programări, rețete, diagnostice, tratamente, trimiteri) filtrate pe: azi, săptămână, lună, an, total.
- CF-12: Administratorul poate adăuga, edita și șterge medici (CRUD complet).
- CF-13: Administratorul poate adăuga, edita și șterge specializări (CRUD complet).
- CF-14: Administratorul poate vizualiza profilul complet al fiecărui medic, inclusiv programările, rețetele, diagnosticele, tratamentele și trimiterile acestuia.

- CF-15: Dashboard-ul admin conține filtre de căutare și sortare pentru entități (medici, specializări, programări etc.).
- CF-16: Rezultatele sunt paginate (implicit 10 rânduri per pagină, configurabil).

Funcționalități Medic

Unui medic îi este permis să:

- CF-17: Medicul gestionează pacienți (adăugare pacient nou sau asociere pacient existent, editare, ștergere).
- CF-18: Medicul vizualizează istoricul medical complet al fiecărui pacient.
- CF-19: Medicul gestionează programări (creare, editare rapidă status, editare completă, ștergere).
- CF-20: Sistemul previne finalizarea unei programări cu dată în viitor.
- CF-21: Sistemul previne marcarea ca 'viitoare' a unei programări cu dată în trecut.
- CF-22: Medicul gestionează diagnostice (CRUD).
- CF-23: Medicul gestionează rețete (CRUD).
- CF-24: Medicul gestionează tratamente (CRUD).
- CF-25: Medicul gestionează trimiteri (CRUD).
- CF-26: Dashboard-ul medicului afișează: solicitări de programare în așteptare, următoarea programare din zi, ultimele 5 programări, programările întârziate.
- CF-27: Dashboard-ul medicului afișează statistici săptămânale (total, finalizate, amânate).
- CF-28: Dashboard-ul afișează rețete și tratamente care expiră în maxim 7 zile.
- CF-29: Medicul poate accepta sau respinge solicitările de programare ale pacienților.
- CF-30: Medicul vizualizează un calendar al programărilor (viitoare, finalizate, amânate).

Funcționalități Pacient

Un pacient poate să execute următoarele acțiuni:

- CF-31: Pacientul vizualizează propriul dosar medical (istoric complet).

- CF-32: Pacientul poate solicita o programare la un medic, specificând interval de date preferat și un mesaj opțional.
- CF-33: Pacientul primește notificări despre statusul solicitărilor (acceptat/respins).
- CF-34: Pacientul poate șterge notificările proprii.
- CF-35: Pacientul primește reminder prin email cu o zi înainte de programare.

Sistem de Email și Notificări

Sistemul de email și notificări îndeplinește următoarele condiții:

- CF-36: Sistemul trimite email automat de confirmare când o programare este acceptată.
- CF-37: Sistemul trimite email automat când o programare este respinsă.
- CF-38: Sistemul trimite remindere automate zilnice (08:00) pentru programări, rețete, tratamente, trimiteri care expiră în curând.

Cerințele Non-Funcționale

Cerințele non-funcționale stabilesc constrângeri și proprietăți ale sistemului. În continuare avem următoarele cerințe:

Securitate

Cerințele de securitate definesc măsurile tehnice implementate pentru a proteja datele medicale sensibile, a preveni accesul neautorizat și a asigura conformitatea cu standardele de securitate specifice domeniului medical.

- CNF-01: Parolele utilizatorilor sunt stocate criptat (bcrypt), niciodată în clar.
- CNF-02: Aplicația este protejată împotriva CSRF (token-uri Laravel).
- CNF-03: Aplicația este protejată împotriva XSS și SQL Injection prin Eloquent ORM.
- CNF-04: Accesul la rute este controlat prin middleware de autentificare (auth) și de rol (rol:admin, rol:medic, rol:pacient).
- CNF-05: Un actor nu poate accesa paginile/datele altui actor (separare strictă a rolurilor).

- CNF-06: Câmpurile sensibile (parola, token remember) sunt excluse din serializare.
- CNF-07: Sesiunile sunt regenerate la autentificare și invalidate la deconectare.
- CNF-08: Câmpurile acceptate prin mass assignment sunt explicit definite (protecție mass assignment vulnerability).

Performanță

Cerințele de performanță stabilesc modul în care aplicația gestionează volumul de date și cererile concurente, asigurând timpi de răspuns acceptabili în condițiile unui flux de lucru medical continuu.

- CNF-09: Interogările sunt paginate (implicit 10 rezultate/pagină) pentru a limita volumul de date transferat.
- CNF-10: Filtrele și sortările se aplică la nivel de query builder (nu în memorie), pentru eficiență.
- CNF-11: Aplicația este optimizată pentru spitale mici și medii; pentru volume mari necesită indexare suplimentară.

Mentenabilitate și Extensibilitate

Cerințele din această categorie stabilesc structura internă a codului și a bazei de date, cu scopul de a facilita întreținerea pe termen lung, integrarea de noi funcționalități și colaborarea eficientă în echipele de dezvoltare.

- CNF-12: Aplicația respectă arhitectura MVC (Model-View-Controller) prin Laravel.
- CNF-13: Baza de date este gestionată prin migrații (rollback, refresh, fresh), fără intervenție manuală.
- CNF-14: Datele de test sunt generate automat prin Seeders (cu Faker, locale ro_RO).
- CNF-15: Arhitectura modulară permite adăugarea de noi module fără impact major.
- CNF-16: Codul este structurat în controllere separate pentru fiecare entitate.

Usabilitate

Cerințele de usabilitate descriu comportamentele interfeței care contribuie la o experiență de utilizare fluidă și intuitivă, reducând erorile de operare și timpul necesar personalului medical pentru efectuarea sarcinilor de rutină.

- CNF-17: Interfața adaptează meniul principal în funcție de rolul utilizatorului autentificat.
- CNF-18: Validarea client-side oferă feedback instant (JavaScript) înainte de trimiterea formularului.
- CNF-19: Mesajele de eroare sunt clare și specifice pentru fiecare câmp.
- CNF-20: Butoanele rapide (finalizare, amânare, ștergere) permit acțiuni frecvente fără navigare suplimentară.

Disponibilitate și Limitări

Această secțiune delimitează explicit condițiile de operare ale aplicației și constrângerile tehnice cunoscute, oferind o imagine realistă asupra scenariilor de utilizare suportate și a celor care necesită soluții suplimentare.

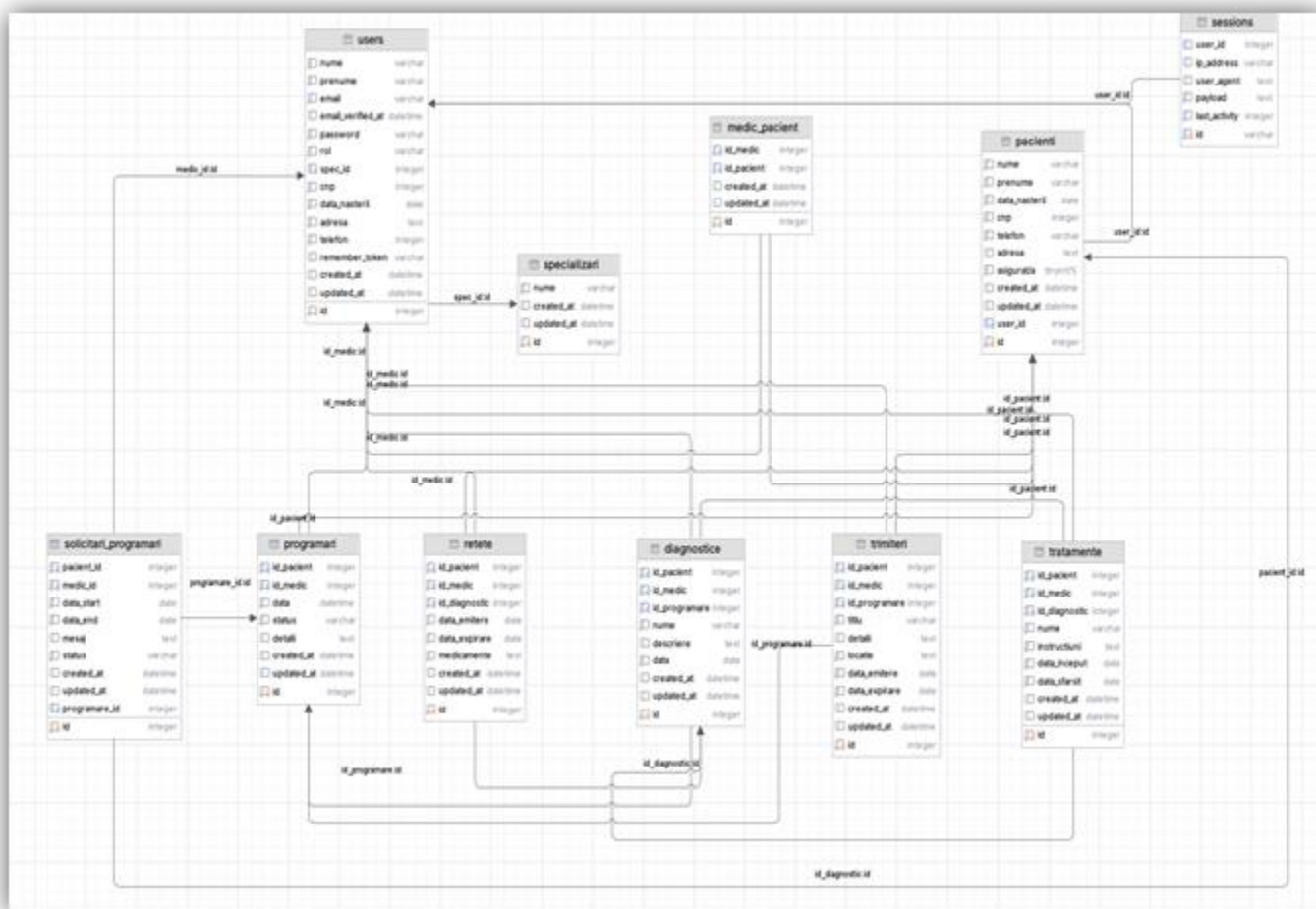
- CNF-21: Aplicația este web-based și necesită conexiune la internet pentru acces.
- CNF-22: Nu suportă mod offline.
- CNF-23: Datele sunt stocate local (control total al spitalului, fără dependență de furnizori cloud).

Arhitectura Aplicației

În acest proiect am implementat o aplicație web ce utilizează framework-ul **Laravel** adaptată necesităților spitalului pentru gestiunea istoricului medical. Sistemul folosește un model de date relațional (MySQL) unde avem entități principale precum **Medic**, **Admin** (cele 2 tipuri de utilizator) și **Pacient**, legate prin relații *one-to-many*. Un admin gestionează medicii, iar un medic gestionează pacienții.

După cum se poate vedea în [figura 10](#), în baza noastră de date avem admini, medici și pacienți, care fiecare la randul lor au programari, diagnostice, rețete, tratamente și trimiteri. Pe lângă acestea, fiecare medic are o specializare, iar legătura dintre medici și

pacienți este făcută cu ajutorul unei tabele pivot (Un medic poate avea mai mulți pacienți, dar și un pacient poate să viziteze mai mulți medici). În urma unei programări rezultă un diagnostic, în urma unui diagnostic rezultă o rețetă și, dacă e cazul, un tratament recomandat de medic. De asemenea, un medic poate să îi ofere unui pacient o trimitere din diverse motive.



Figură 10 - Diagrama Entitate-Relație a aplicației E-Spital

În acest fel, un medic poate avea mai mulți pacienți, un pacient poate avea mai mulți medici, un medic poate avea o singură specializare însă o specializare poate aparține mai multor medici. Un medic/pacient poate avea mai multe programări, diagnostice, rețete, tratamente și trimiteri însă nu și invers. O programare poate avea mai multe diagnostice sau trimiteri, un diagnostic poate aparține mai multor rețete sau tratamente,

o rețetă/tratament nu poate avea mai multe diagnostice. Pacientii pot solicita mai multe programări, iar medicii pot primi, de asemenea, mai multe solicitări.

În Laravel, relațiile menționate anterior sunt create cu ajutorul modelelor. **Modelul** are rolul de a prelua datele din baza de date, făcându-le ușor accesibile în cadrul aplicației.

```
class User extends Authenticatable {
    protected $fillable = ['nume', 'prenume', 'email', 'password',
        'rol', 'spec_id', 'cnp', 'data_nasterii', 'adresa', 'telefon',];

    public function isAdmin() { return $this->rol === 'admin'; }
    public function isMedic() { return $this->rol === 'medic'; }
    public function isPacient(){ return $this->rol === 'pacient';}

    public function specializare() {
        return $this->belongsTo(Specializare::class, 'spec_id'); }
    public function pacienti() {
        return $this->hasMany(Pacient::class, 'medic_pacient',
            'id_medic', 'id_pacient'); }
    public function programari() {
        return $this->hasMany(Programare::class, 'id_medic'); }
    public function retete() {
        return $this->hasMany(Reteta::class, 'id_medic');}
    public function diagnostice() {
        return $this->hasMany(Diagnostic::class, 'id_medic');}
    public function tratamente() {
        return $this->hasMany(Tratament::class, 'id_medic');}
    public function trimiteri() {
        return $this->hasMany(Trimitere::class, 'id_medic');}
    public function solicitariPrimate() {
        return $this->hasMany(SolicitareProgramare::class, 'medic_id'); }
    public function pacient() {
        return $this->hasOne(Pacient::class); } }
```

Continuând cu user-ul, pentru acesta am creat un model. Proprietatea \$fillable are rolul de a specifica exact ce câmpuri pot fi completate automat atunci când se creează sau se actualizează un model prin metode precum create() sau update(). Prin acest

mod, noi protejăm aplicația împotriva atribuirii în masă (mass assignment vulnerability), căci nu vrem ca orice câmp din request (cum ar fi `is_admin` sau `id`) să fie inserat automat.

Prin funcțiile `is_admin` și `is_medic` verificăm rolul user-ului pentru a-i oferi mai departe privilegiile lui respective. Pentru user-ul care are rolul de medic, am creat relațiile specifice lui.

De asemenea, parolele sunt protejate (acestea sunt obfuscate).

Baza de date a fost creată cu ajutorul migrărilor (**migrations**). Acestea creează tabelele fără a mai fi necesar să le creăm în mod manual. Așa cum este specificat în documentația oficială, „*fiecare nume de fișier de migrare conține un timestamp care permite Laravel să determine ordinea migrărilor*: `php artisan make:migration create_flights_table` Dacă Laravel poate determina numele tabelului din numele migrării, Laravel va precompleta fișierul de migrare generat cu tabelul specificat.”. [22] Această metodă dispune de anumite avantaje, dispunând de mijloace avansate de control asupra bazei de date precum anularea modificărilor(migrărilor) efectuate (rollback), rollback la toate migrările si apoi remigrare (refresh), ștergerea tuturor tabelelor si apoi remigrare (fresh), etc.

Mai jos avem un exemplu de migrare pentru useri:

```
return new class extends Migration
{
    function up(): void
    {
        Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('nume');
            $table->string('prenume');
            $table->string('email')->unique();
            $table->timestamp('email_verified_at')->nullable();
            $table->string('password');
            $table->enum('rol', ['admin', 'medic', 'pacient'])->default('medic');
            $table->integer('spec_id')->nullable();
        });
    }
}
```

```
$table->foreign('spec_id')->references('id')->on('specializari')->onDelete('set null');
$table->integer('cnp');
$table->date('data_nasterii');
$table->text('adresa');
$table->integer('telefon');
$table->rememberToken();
$table->timestamps();
});
```

Se poate vedea modul în care sunt create atributele tabelii. Userii sunt împărțiți în medici, pacienți și admini, iar dacă user-ul este medic, îi este atribuit și o specializare.

View-urile reprezintă interfața cu utilizatorul, paginile în sine, pagini care afișează informații. Un view primește date de la Model (prin Controller) și le afișează. Un view este un fișier template scris într-un limbaj specific Laravel numit *Blade*. Documentația oficială specifică faptul că „*view-urile Blade pot fi returnate de rute sau controllere folosind ajutorul pentru view-uri globale.*” [22]

În termeni mai simplii, aceste view-uri, odată create, trebuie definite în rute (**Routes**). Prin rute noi specificăm legătura dintre o adresă URL și o acțiune din aplicație. Pe scurt, ele spun framework-ului ce să facă atunci când un utilizator accesează o anumită adresă web. Documentația Laravel precizează că „*toate rutele Laravel sunt definite în fișierele de rută, care se află în directorul `routes`*“, că „*fișierul `routes/web.php` definește rutele destinate interfeței web*” precum și faptul că „*rutele definite în `routes/web.php` pot fi accesate prin introducerea adresei URL a rutei definite în browser.*” [22]

Mai exact, scopul lor este să:

1. Facă legătura între cererile HTTP (GET, POST, PUT, DELETE) și funcțiile (metodele) din Controller care le procesează.
2. Controleze navigarea în aplicație (ce pagină vede utilizatorul).
3. Asigure protecția accesului (prin *middleware*, ex: autentificare sau roluri).

4. Definească resurse RESTful (ex: CRUD automat pentru medici, pacienți etc.).
5. Simplifice codul, astfel încât să nu mai fie nevoie să scrii manual logica de rutare.

În aplicația noastră avem:

```
Route::view('/', 'index')->name('index');
Route::redirect('/', '/login');
Route::post('/login', [AuthController::class, 'login']);
// ...alte rute pentru login și logout

Route::middleware(['auth', 'rol:admin'])->group(function () {
    Route::get('/admin/dashboard', [AdminController::class, 'index'])->name('admin.dashboard');
});

// ... dashboard și pentru medici, pacienți
Route::middleware(['auth', 'rol:admin'])->group(function () {
    Route::resource('medici', MedicController::class);
    Route::resource('specializari', SpecializareController::class);
});

Route::middleware(['auth', 'rol:medic'])->group(function () {
    Route::resource('pacienti', PacientController::class);
    // ...programări, rețete, etc...});

Route::middleware(['auth'])->group(function () {
    Route::get('/cont', [ContController::class, 'show'])->name('cont');// ...});

Route::middleware(['auth'])->group(function () {
    // ...rute pentru solicitări...
});
```

Când un utilizator accesează o adresă, cererea HTTP ajunge la Router, Routerul caută o definiție care corespunde acelei adrese, și găsește ruta: `Route::get('/login', [AuthController::class, 'showLogin'])`;; apoi Laravel execută metoda

`showLogin()` din `AuthController`, ca mai apoi acea metodă să returneze o vedere (View) sau să redirecționeze către altă rută.

Dacă avem decât rute simple, precum `Route::view('/', 'index')->name('index');`, când accesăm `/`, Laravel returnează direct view-ul *index.blade.php*.

Dacă avem rute pentru autentificare, precum `Route::get('/login', [AuthController::class, 'showLogin'])->name('login');` și `Route::post('/login', [AuthController::class, 'login']);`, cea cu metoda GET afișează formularul de logare iar cea cu metoda POST procesează datele din formular și verifică user-ul.

Middleware-ul din cadrul rutelor are rolul de a oferi protecția datelor. În cazul nostru, pagina din cadrul `Route::middleware(['auth', 'rol:admin'])->group(function () { Route::get('/admin/dashboard', [AdminController::class, 'index'])->name('admin.dashboard');});` este accesibilă numai dacă utilizatorul este autentificat și are rolul de admin. Middleware-ul *auth* verifică dacă userul este logat, iar *rol:admin* dacă are permisiunea potrivită.

Mai avem rute de tip Resource, precum `Route::resource('medici', MedicController::class);`, care creează automat 7 rute RESTful pentru operații CRUD (Create, Read, Update, Delete):

Metodă HTTP	URL	Acțiune	Funcție Controller
GET	/medici	listare	index()
GET	/medici/create	formular creare	create()

POST	/medici	salvare nou	store()
GET	/medici/{id}	afișare detalii	show()
GET	/medici/{id}/edit	formular edit	edit()
PUT/PATCH	/medici/{id}	actualizare	update()
DELETE	/medici/{id}	ștergere	destroy()

Avem și rute accesibile doar de utilizatorii logați, pentru vizualizarea și modificarea contului, oferind încă un mijloc de protejare a datelor prin Middleware.

După cum se poate vedea din rute, au fost create multe view-uri (pagini) accesibile atât de medici, cât și de admini, dar protejate: un medic nu poate vedea paginile(și astfel datele și informațiile) unui admin, dar nici un admin nu poate vedea paginile unui medic.

Pentru paginile noastre, Laravel dispune de mai multe funcționalități folositoare. De exemplu, toate paginile din această aplicație au o anumită structură, care, în mod normal, ar fi trebuit repetată mereu de fiecare dată când creăm o nouă pagină. Această structură repetitivă poartă numele de *layout*, iar în Laravel nu este necesar decât o singură dată să stabilim layout-ul paginilor noastre. Astfel, în folder-ul *layouts* am creat o pagină specială care afișează elementele repetitive din paginile noastre: header-ul și footer-ul:

```
<!DOCTYPE html>
// ...
@auth
```

```

@if(auth()->user()->rol == 'medic')
<li><a href="{{ route('programari.index') }}">Programări |</a></li>
<li><a href="{{ route('pacienti.index') }}">Pacienți |</a></li>
//...
<li><a href="{{ route('medic.dashboard') }}">Dashboard |</a></li>
<li><a href="{{ route('cont') }}">Cont</a></li>
@elseif (auth()->user()->rol == 'admin')
//...
@elseif(auth()->user()->rol == 'pacient')
//...
@endif
@endauth
@yield('content')
<div class="footer">
&copy; 2026 - Spitalul Life. Toate drepturile rezervate.
</div>

```

Se poate observa cum header-ul se schimbă în funcție de cine este autentificat.

Acuma, de fiecare dată când creăm o pagină, nu trebuie decât să specificăm acest lucru:

```

@extends('layouts.app')
@section('content')
...pagina noastră...
@endsection

```

Laravel poate popula baza noastră de date în mod automat cu ajutorul Seederi-ilor. Un seeder creează date false, cu scopul de a testa aplicația înainte de a fi lansată pentru public.

Un exemplu de seeder din aplicația noastră este:

```

class MediciSeeder extends Seeder {
    public function run(): void
    {
        User::create([
            'nume'=>'Popescu',
            'prenume'=>'Ion',
            'email'=>'pop@yahoo.com',

```

```
        'password'=>'popescu',  
        // ...  
]);  
$faker = Faker::create('ro_RO');  
for ($i=0; $i<8; $i++) {  
    User::create([  
        'nume' => $faker->lastName,  
        'prenume' => $faker->firstName,  
        'email' => $faker->unique()->safeEmail,  
        'password' => Hash::make('password'),  
        // ...  
    ])  
}
```

În mod concret, acesta creează în mod explicit un anume medic Popescu cu anumite date (ca dezvoltatorul să poată testa paginile accesibile de un medic, altfel nu ar putea nici dacă s-ar uita în baza de date căci parolele sunt hash-uite). Apoi, creează în mod automat și total aleatoriu încă alți 9 medici cu date specifice românești (și de aici parametrul `ro_RO`, altfel by default datele create sunt în engleză).

Controller-ele reprezintă logica din spatele aplicației, intermediarul dintre View și Model. Acestea pot primi cereri de la utilizator și pot decide ce View să afișeze și ce date să preia sau să modifice prin Model. Printre exemple, se numără metode care răspund la rute HTTP, procesează formulare, apelează modelul și trimite rezultatele către View.

Pentru această aplicație am stabilit controllere care să gestioneze procesul de autentificare, contul utilizatorului logat dar și fiecare entitate în parte.

Pentru autentificarea pe site, am creat un formular, unde utilizatorul își introduce e-mail-ul și parola. Dacă aceste date sunt incorecte, utilizatorul va primi un mesaj de eroare specific (de exemplu, parolă incorectă, sau parola trebuie să conțină cel puțin 6 caractere)

Spitalul Life

Inregistrare cont nou

Nume:

Prenume:

Email:

Parola:

Roll:

Specializarea (daca este cazul):

CNP:

Adresa:

Telefon:

Figură 11- Pagina unde utilizatorul se poate înregistra în sistem

Dacă acesta nu și-a creat un cont până în acel moment, o poate face prin Sign up. Când utilizatorul apasă pe butonul respectiv, este trimis pe o altă pagină, ci anume `register.blade.php`, așa cum se poate vedea și pe [figura 11](#).

```
<button type="button" class="form-button"
onclick="location.href='register'">Sign up</button>
```

Logica din spatele înregistrării este aplicată în controller:

Metoda `showRegister()` afișează formularul de înregistrare, adică încarcă view-ul în sine, și preia toate specializările din tabelul respectiv pentru a le afișa într-un dropdown (necesar dacă se înregistrează un medic).

```
public function showRegister() {
    $specializari = Specializare::all();
    return view('register', compact('specializari'));}
```

Metoda `register(Request $request)` procesează formularul de înregistrare. Inițial, avem niște reguli, care verifică corectitudinea câmpurilor completate de utilizator.

```
public function register(Request $request) {
    $rules = [
        'nume' => 'required|string|max:20',
        'prenume' => 'required|string|max:100',
        'email' => 'required|email|unique:users,email|max:50',
        // ...
    ];
```

De asemenea, se verifică dacă utilizatorul își creează contul ca medic, admin sau pacient, și aplică și acolo reguli care să verifice input-ul, dacă este cazul, altfel setăm acel atribut ca fiind nul. În altă ordine de idei, dacă rolul ales este medic, atunci trebuie să aibă o specializare validă (care există în baza de date), însă dacă e admin, câmpul `spec_id` nu e obligatoriu. Dacă este pacient, acestuia trebuie să i se verifice faptul dacă are o asigurare.

```
if ($request->rol === 'medic') {
    $rules['spec_id'] = 'required|exists:specializari,id';
}
else {
    $rules['spec_id'] = 'nullable';
}
if ($request->rol === 'pacient') {
    $rules['asigurat/a'] = 'required|boolean';
}
```

```
else {  
    $rules['asigurat/a'] = 'nullable';  
}
```

Mai departe, pentru aceste reguli, dacă ele nu sunt respectate, trebuie să afișăm utilizatorului mai exact ce date au fost introduse incorect.

```
$data = $request->validate($rules, [  
    'nume.required' => 'Numele este obligatoriu.',  
    'nume.string' => 'Introduceti litere.',  
    'nume.max' => 'Numele nu poate contine mai mult de 20 de caractere.',  
    'prenume.required' => 'Prenumele este obligatoriu.',  
    'prenume.string' => 'Introduceti litere.', // ... ]));
```

Parolele trebuiesc a fi criptate, prin hash-ing, pentru securitate, apoi utilizatorul este salvat în baza de date cu ajutorul modelului *User*.

```
$data['password'] = Hash::make($data['password']);  
$user = User::create($data);
```

După înregistrare, utilizatorul este conectat automat (fără să mai dea login) și redirecționat către dashboard-ul potrivit.

```
Auth::login($user);  
if (Auth::user()->isAdmin()) {  
    return redirect()->route('admin.dashboard');  
}  
if (Auth::user()->isMedic()) {  
    return redirect()->route('medic.dashboard');  
}  
if (Auth::user()->isPacient()) {  
    return redirect()->route('pacient.dashboard');  
}}}
```

Pentru partea de logare, la fel ca la pagina de înregistrare, avem aceeași metoda pentru afișare:

```
public function showLogin() { return view('login'); }
```

Pentru logarea în sine, avem:

```
public function login(Request $request) {  
    $date = $request->validate([  
        'email' => 'required|email',  
        'password' => 'required|min:6',]);  
    if (Auth::attempt($date)) {  
        request->session()->regenerate();  
        if (Auth::user()->isAdmin()) {  
            return redirect()->route('admin.dashboard'); }  
        // ...}  
    return back()->withErrors(['email' => 'Email sau parolă  
incorectă.']); }
```

Validează sub aceeași manieră datele introduse, apoi, `Auth::attempt($date)` verifică în baza de date dacă există un utilizator cu acel email și parolă și regenerează sesiunea pentru securitate dacă acesta există. Se verifică rolurile pentru a trimite utilizatorul pe pagina corectă, însă dacă datele sunt greșite afișează un mesaj de eroare.

În final, pentru delogare, avem:

```
public function logout(Request $request) {  
    Auth::logout();  
    $request->session()->invalidate();  
    $request->session()->regenerateToken();  
    return redirect()->route('login'); }
```

Aici, metoda dată deconectează utilizatorul curent, invalidează sesiunea curentă (adică șterge datele), regenerează tokenul CSRF pentru securitate și redirecționează către pagina de login.

Datele le verificăm atât server-side (prin controller-ul prezentat) cât și client-side, prin Javascript, pentru un plus de securitate, folosind o funcție aplicată la formular:

```
function validForm() {  
    const email = document.getElementById('email').value.trim();
```



```

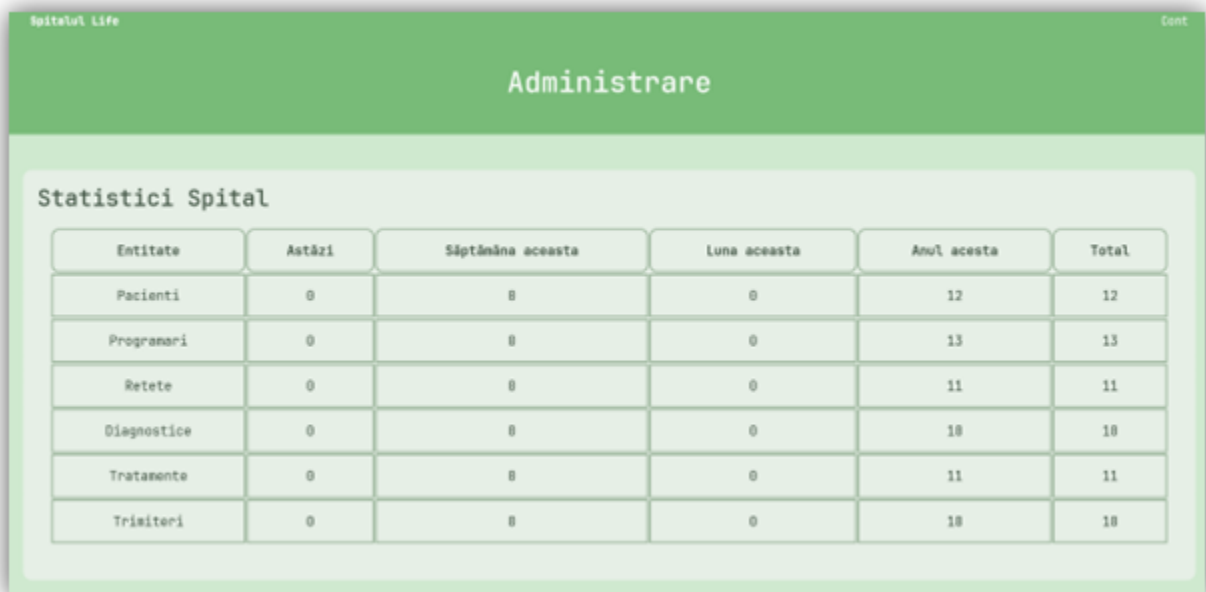
const password = document.getElementById('password').value.trim();

if (!email) { alert("Introduceti emailul!"); return false; }
if (!password) { alert("Introduceti parola!"); return false; }
if (password.length < 6) { alert("Parola trebuie să aibă cel puțin 6 caractere!"); return false; }
return true;

```

Astfel, selectăm elementele din pagina HTML și verificăm dacă aceste câmpuri au fost introduse sau dacă respectă anumite specificații (cum ar fi lungimea parolei).

Odată autentificat, user-ul este trimis fie în pagina principală de administrator, fie în pagina principală de medic.

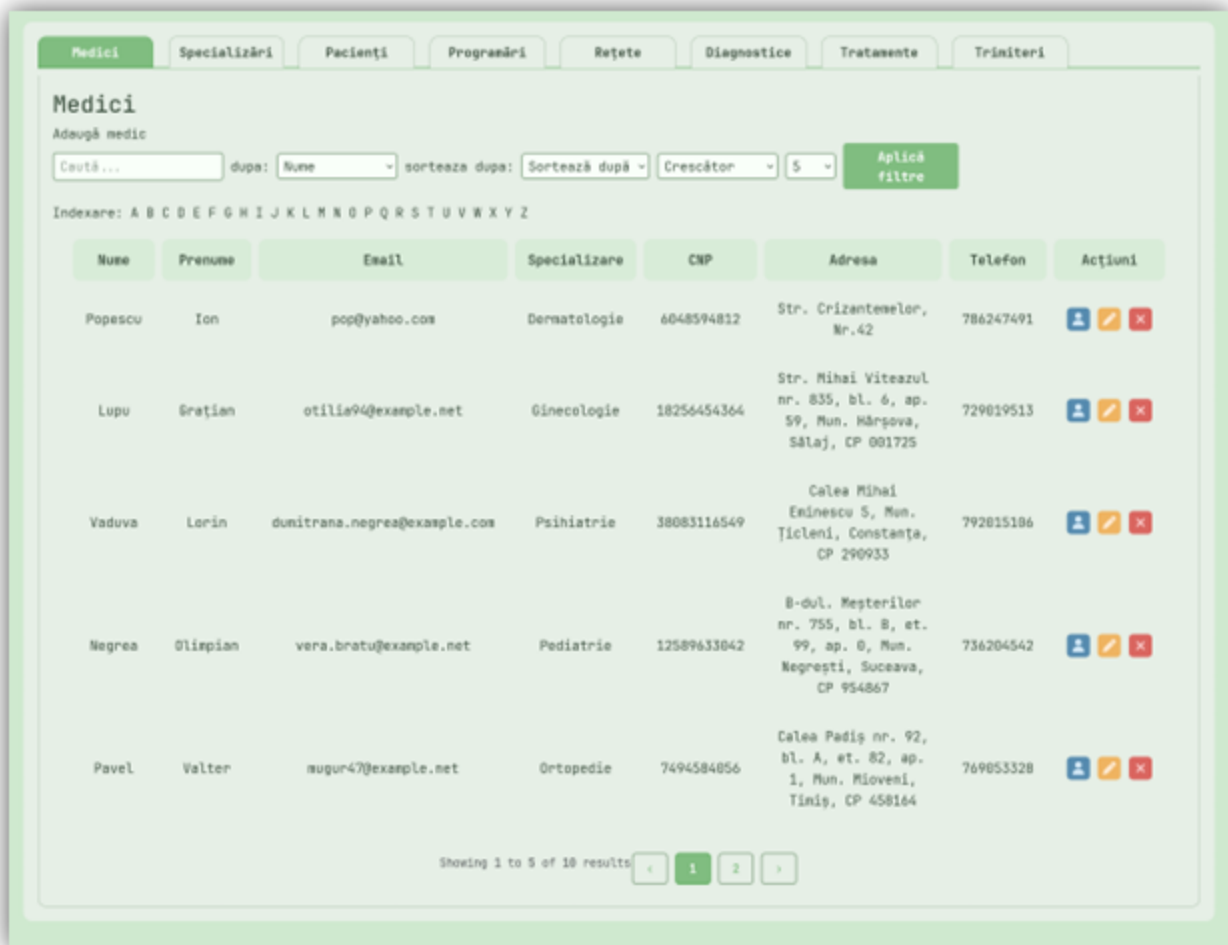


The screenshot shows a web application interface for 'Spitalul Life'. The top navigation bar is green with the text 'Spitalul Life' on the left and 'Cont' on the right. Below the navigation bar is a green header with the word 'Administrare' in white. The main content area has a light green background and contains a section titled 'Statistici Spital'. This section features a table with 6 columns: 'Entitate', 'Astăzi', 'Săptămâna aceasta', 'Luna aceasta', 'Anul acesta', and 'Total'. The table contains 6 rows of data for different hospital entities.

Entitate	Astăzi	Săptămâna aceasta	Luna aceasta	Anul acesta	Total
Pacienti	0	0	0	12	12
Programari	0	0	0	13	13
Retete	0	0	0	11	11
Diagnostiche	0	0	0	10	10
Tratamente	0	0	0	11	11
Trisiteri	0	0	0	10	10

Figură 12 – Pagina principală pentru un administrator, secțiunea de statistici

Ca și administrator, putem să vizualizăm datele întregului spital, dar să și modificăm medicii cu specializările lor.



Figură 13 – Pagina principală pentru un administrator, secțiunea de modificare a medicilor și vizualizare date spital

Logica pentru efectuarea acestor acțiuni, precum și pentru vizualizarea tuturor acestor date, se află în AdminController.

Pentru afișarea statisticilor mai întâi preluăm intervalele de timp, adică începutul fiecărei zi, săptămâni, luni și an.

Funcția `$statsFor` calculează, pentru orice model trimis, numărul de înregistrări create în ziua curentă, de la începutul săptămânii, de la începutul lunii și de la începutul anului până în momentul actual, dar și numărul total de înregistrări în tabel.

```
$statsFor = function ($model) use ($today, $weekStart, $monthStart,
$yearStart) {
return [
```

```

    'today' => $model::whereDate('created_at', $today)->count(),
    'week'  => $model::whereBetween('created_at', [$weekStart,
Carbon::now()])->count(),
    // ...
    'total' => $model::count(),];};

```

Astfel, această funcție va fi aplicată asupra modelelor:

```

$stats = ['pacienti' => $statsFor(Pacient::class), // ... ];

```

După tabelul cu statistici, în dashboard avem un tabel cu tab-uri și filtre de căutare și sortare. Pentru fiecare entitate am aplicat logica pentru a gestiona căutarea cu filtre și sortare. Un exemplu aplicat avem pentru programări.

Creăm un query builder pentru tabelul nostru și încărcăm relația pacient, apoi aplicăm filtrele de căutare după numele pacientului și a medicului, după dată și statusul programării (viitoare, finalizată, amânată).

```

$specQuery = Specializare::query();
if ($request->filled('nume')) {
    $specQuery->where('nume', 'like', '%' . $request->nume . '%');
}
if ($request->filled('litera')) {
    $specQuery->where('nume', 'like', $request->litera . '%');
}
if ($request->filled('direction')) {
    $specQuery->orderBy('nume', $request->direction);
}
$specLimit = $request->get('limit', 10);
$specializari = $specQuery->paginate($specLimit)->withQueryString();

```

Secvența `whereHas('pacient'/'medic', ...)` filtrează programările doar pe cele care au pacientul sau medicul potrivit. Se caută programările unde pacientul are numele sau prenumele care conține textul introdus.

Filtrele pentru dată și status filtrează programările exact pe data sau statusul selectat.

Mai departe avem sortarea după aceste date:

```
if ($request->filled('sort_by') && in_array($request->sort_by,
['nume','data']))
{
    if ($request->sort_by === 'nume') {
        $progQuery->join('pacienti', 'pacienti.id', '=',
'programari.id_pacient')->orderBy('pacienti.nume', $request-
>direction ?? 'asc')->select('programari.*');
    }
    elseif ($request->sort_by === 'medic') {
        $progQuery->join('users as medici', 'medici.id', '=',
'programari.id_medic')->orderBy('medici.nume', $request->direction ??
'asc')->select('programari.*');
    }
    else {
        $progQuery->orderBy('data', $request->direction ?? 'asc');}}

```

Se verifică dacă userul a cerut sortare, apoi se sortează după criteriul ales.

Pentru sortarea după numele pacientului, join-ul alătură tabelul pacienți la tabelul nostru cu programări, iar apoi ordonăm după numele pacientului din acel tabel, și în final selectăm doar coloanele din tabelul programări.

Aceeași sortare se aplică și pentru medici numai că folosim tabelul users pentru aceștia.

Dacă nu se cere nici nume pacient, nici nume medic, sortarea se face după dată.

Pentru paginare și afișarea limitată a datelor avem:

```
$today = now();
$progLimit = $request->get('limit', 10);
$programari = $progQuery->paginate($progLimit)->withQueryString();

```

Variabila `$progLimit` semnifică câte rezultate se vor afișa pe pagină (10 rânduri în mod implicit). *Paginate* înseamnă că noi dorim ca aceste rezultate să fie împărțite pe mai multe pagini, iar `withQueryString()` păstrează filtrele în URL la schimbarea paginii (ca să nu se piardă sortarea / filtrarea).

În tabelele cu medicii și specializările noi putem să adăugăm medici sau specializări noi, să vizualizăm profilul fiecărui medic și să edităm pe pagini separate, dar să și îi stergem. Această logică este aplicată în `MedicController` pentru medici și `SpecializareController` pentru specializări prin operații CRUD.

Pentru `SpecializareController` avem următoarele operații:

`index()` – Afișează dashboard-ul adminului și trimite în view toți userii care au rolul „medic” și lista completă de specializări

`create()` – Afișează formularul de creare specializare și trimite în view aceeași listă ca la `index()`.

`store()` – Salvează o specializare nouă validând câmpul cu numele specializării, creând specializare în baza de date și redirecționând înapoi cu un mesaj de succes:

`edit()` – Afișează pagina de editare găsind specializarea și trimițând obiectul în view la formularul de editare.

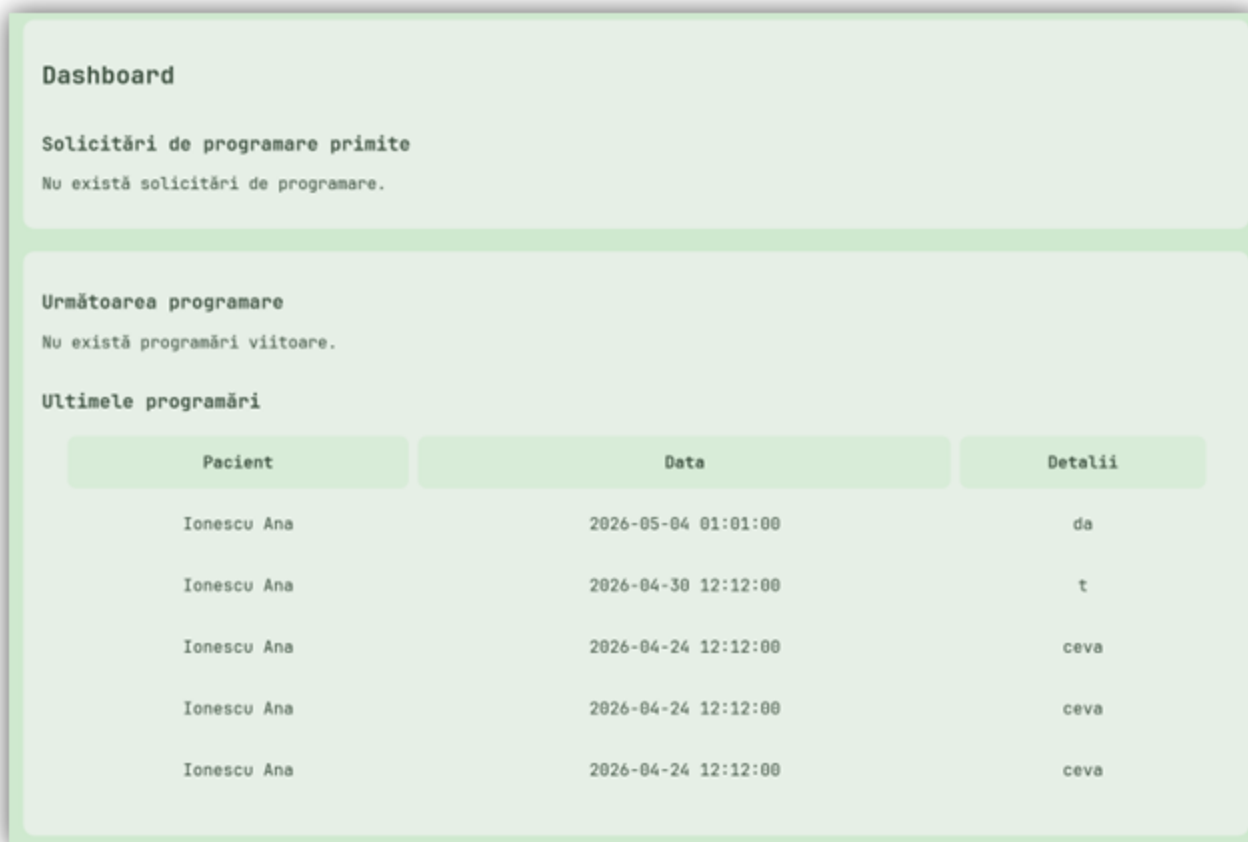
`update()` – Actualizează o specializare validând noul nume.

`destroy()` – Șterge o specializare găsind specializarea, ștergând-o și trimițând mesaj de succes.

Aceleași operații sunt aplicate pe toate entitățile.

Administratorul poate vedea și profilul fiecărui medic (datele personale și toate programările, rețetele, diagnosticele, tratamentele și trimerile pe care le are). Revenind la `MedicController` avem metoda `show()` care afișează toate aceste date.

Logându-ne ca și medic avem acum pagina principală a medicului (medic dashboard), unde avem afișate diverse informații utile.



Figură 14 – Pagina principală pentru un medic, secțiunea notificărilor a solicitărilor și a programărilor

Meniul principal conține butoanele care trimit către paginile principale ale unui medic în care gestionează programările, rețetele, pacienții, diagnosticele, trimerile și tratamentele.

În restul dashboard-ului avem afișate câteva informații rapide și de folos.

În primul rând, avem afișate solicitările pacienților pentru programare. Pacienții pot să își aleagă o dată preferabilă pentru programare și să solicite o programare, iar medicul poate să decidă dacă dorește să pună o programare în perioada respectivă sau să respingă din diverse motive personale.

În al doilea rând, avem afișate următoarele programări ale unui medic în ziua respectivă, precum și ultimele 5 programări efectuate, dar și cele întârziate.

```
$medicId = auth()->id();
```

```
$nextProgram = Programare::where('id_medic', $medicId)->where('data',  
'>=', now())->orderBy('data', 'asc')->first();  
// ...
```

Pentru a afișa următoarea programare, se caută programările medicului, unde există doar doar cele începând de acum încolo, se sortează cele mai apropiate primele, și se ia prima din listă (adică cea mai apropiată programare viitoare).

Pentru a afișa ultimele 5 programări, se caută din programările medicului doar pe cele din trecut, se sortează după dată de la cea mai recentă la cea mai veche programare, apoi se iau doar primele 5 și se returnează lista lor.

Pentru a afișa programările cu întârziere (adică programări din trecut care nu au fost trecute ca fiind finalizate), se caută programările medicului care au ca și status “viitoare”, dar la care data programării este deja depășită.

În continuare, avem statisticile pentru săptămâna curentă:

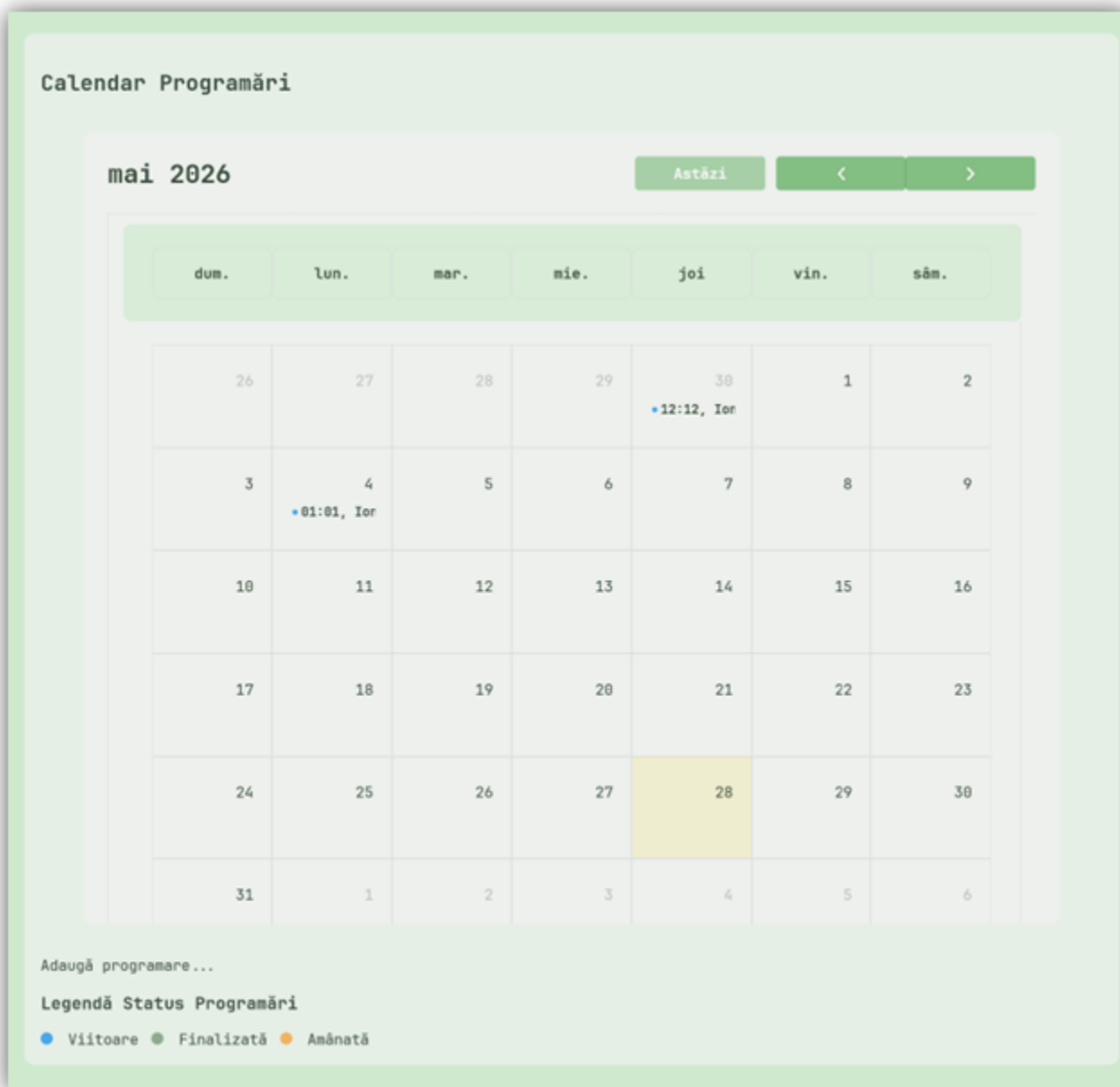
```
// ...  
$stats = [  
'total' => $weekPrograms->count(),  
'finalizate'=> $weekPrograms->where('status', 'finalizata')->count(),  
'amanate' => $weekPrograms->where('status', 'amanata')->count(),];
```

Pentru a selecta programările din săptămâna curentă, se preiau data și ora actuală, se setează data la prima zi a săptămânii, dar și la ultima zi a săptămânii, apoi cu acestea se obține toate programările medicului pentru săptămâna curentă. Apoi, pentru a putea afișa totalul tuturor programărilor nu facem nimic altceva în plus decât să le numărăm, iar pentru cele amânate și cele finalizate mai specificăm și statusul lor în plus.

Acum, nu mai avem altceva decât rețetele și tratamentele care vor expira în maxim o săptămână. Acestea au fost implementate într-o manieră asemănătoare. Se extrag din baza de date atât rețetele, cât și tratamentele unui medic care expiră între astăzi și peste 7 zile, încărcând și pacientul cu diagnosticul asociată rețetei sau tratamentului.

Mai departe, un medic trebuie să gestioneze separat programările, rețetele, tratamentele și trimerile.

Pentru programări, avem următoarea pagină:



Figură 15 – Pagina de programări, secțiunea calendar

Avem un calendar care afișează programările viitoare, finalizate și amânate, dar și liste unde putem de asemenea filtra căutări. Se poate observa cum putem edita programările viitoare în mod direct: le putem finaliza (butonul verde), amâna (butonul galben), sau

șterge (butonul roșu) instant, dar putem să și edităm datele mai amănunțit (butonul portocaliu).

Filtrează Programările...
sorteaza dupa:

Sortează după
Crescător
5

Aplică filtre

Programări viitoare

Data	Pacient	Detalii	Acțiuni
24.04.2026, 12:12	Ionescu Ana	ceva	☒ ☐ ☐ ☐ ☐
30.04.2026, 12:12	Ionescu Ana	t	☒ ☐ ☐ ☐ ☐
16.04.2026, 12:12	Ionescu Ana	e	☒ ☐ ☐ ☐ ☐
16.04.2026, 12:12	Ionescu Ana	e	☒ ☐ ☐ ☐ ☐

Programări trecute

Data	Pacient	Detalii
18.03.2026, 23:09	Anton Ernest	Similique qui debitis occaecati minima.
11.03.2026, 12:12	Ionescu Ana	ok
24.04.2026, 12:12	Ionescu Ana	ceva

Programări amanate

Data	Pacient	Detalii
Nu există programări amanate.		

Figură 16 – Pagina programărilor, secțiunea de gestiune al acestora

În acest fel, dacă edităm programarea doar prin butoanele rapide, avem:

```
public function update(Request $request, string $id) {
    $programare = Programare::findOrFail($id);
    if ($request->has('status') && !$request->has('id_pacient')) {
        $request->validate([...]);
        if ($request->status === 'finalizata' && $programare->data-
            >isFuture()) { return redirect()->back()->withErrors([...]); }
        $programare->update(['status' => $request->status,]);
        return redirect()->route('programari.index')
    }
}
```

Adică, dacă cererea are decât câmpul status și nu conține și pacientul, atunci nu edităm decât statusul în sine. De asemenea, dacă medicul încearcă să seteze o programare ca finalizată, dar data programării este în viitor, atunci va primi un mesaj de eroare. Apoi programarea este actualizată în baza de date iar medicul este redirecționat înapoi.

Însă, dacă medicul dorește să modifice celălalte câmpuri dintr-o programare, cum ar fi numele pacientului, sau date, atunci avem și această opțiune.

Se validează datele, apoi și aici verificăm statusul programării: putem finaliza o programare doar dacă data ei este în trecut, sau dacă statusul este pentru o programare viitoare, data trebuie să fie în viitor. În urma acestor verificări se actualizează programarea în baza de date și se redirecționează înapoi.

Pentru rețete, diagnostice, tratamente și trimiteri avem un format asemănător, unde le putem crea, șterge și edita folosind logică asemănătoare (CRUD). Pentru pacienți, însă, avem un lucru în plus: fiecare pacient are propriul său profil (așa cum și fiecare medic avea propriul său profil pentru un administrator, prezentat anterior în [figura 13](#)).

Când accesăm butonul de profil al unui pacient, putem vedea datele personale ale acestuia, toate programările, rețetele, diagnosticele, tratamentele și trimiterile efectuate

asupra acestui pacient de către medicul logat, precum și istoricul complet al pacientului respectiv.

Ca și pacient, avem o pagină unde avem afișate notificările pacientului, istoricul acestuia, precum și informații relevante despre spital. Acesta poate solicita o programare, unde își poate alege o dată, precum și un medic.



Figură 17 – Pagina pacientului, secțiunea de notificări

Avem integrat un sistem de notificări, în care primim răspunsul dat în urma unei solicitări de programare de către un medic. Pentru efectuarea acestui lucru, am creat o tabelă intermediară, `SolicitareProgramare`, care reține numele pacientului care solicită, a medicului solicitat, a datei preferate de pacient, un mesaj, și poate avea 3 stări: trimisă, rezolvată, precum și respinsă.

Desigur că logica din spate se află în controllere specifice: `PatientSolicitareController` și `MedicSolicitareController`.

În `PatientSolicitareController` trimitem către pagina unde pacientul solicită programare, validăm datele introduse și creăm solicitarea în sine.

În `MedicSolicitareController`, nu facem altceva decât să gestionăm cazul în care medicul refuză solicitarea, căci dacă acesta navighează către crearea programării, logica va fi gestionată de controller-ul de la programări, fiind în esență același lucru, însă ușor

mai rapid, căci nu mai este necesar să navigheze către pagina de programări și apoi cea de creare de programări. Dacă medicul refuză solicitarea, starea solicitării se va schimba

Pentru acest sistem de notificări am adăugat și notificări prin e-mail, împreună cu remindere cu o zi înainte pentru programări și expirarea rețetelor, tratamentelor și trimiterilor.

De exemplu, pentru programări, a fost necesar să facem pagina blade în sine pentru mail, logica pentru mail (realizată prin `php:artisan make:mail`) și comanda care gestionează trimiterea email-ului (`Console/Commands/TrimiteReminderProgramari.php`).

Avem 3 tipuri de pagini blade: pentru cazul în care programarea este acceptată de medic, pentru cazul în care este respinsă, și pentru reminder.

Exemplu pentru `acceptate.blade.php`:

```
@component('mail::message')

# Programare acceptată!
Bună ziua, **{{ $programare->pacient->prenume }}**!
Programarea dumneavoastră a fost **acceptată**.
```

```
### Detalii programare
- **Data:** {{ $programare->data }}
- **Medic:** {{ $programare->medic->nume }} {{ $programare->medic->prenume }}
```

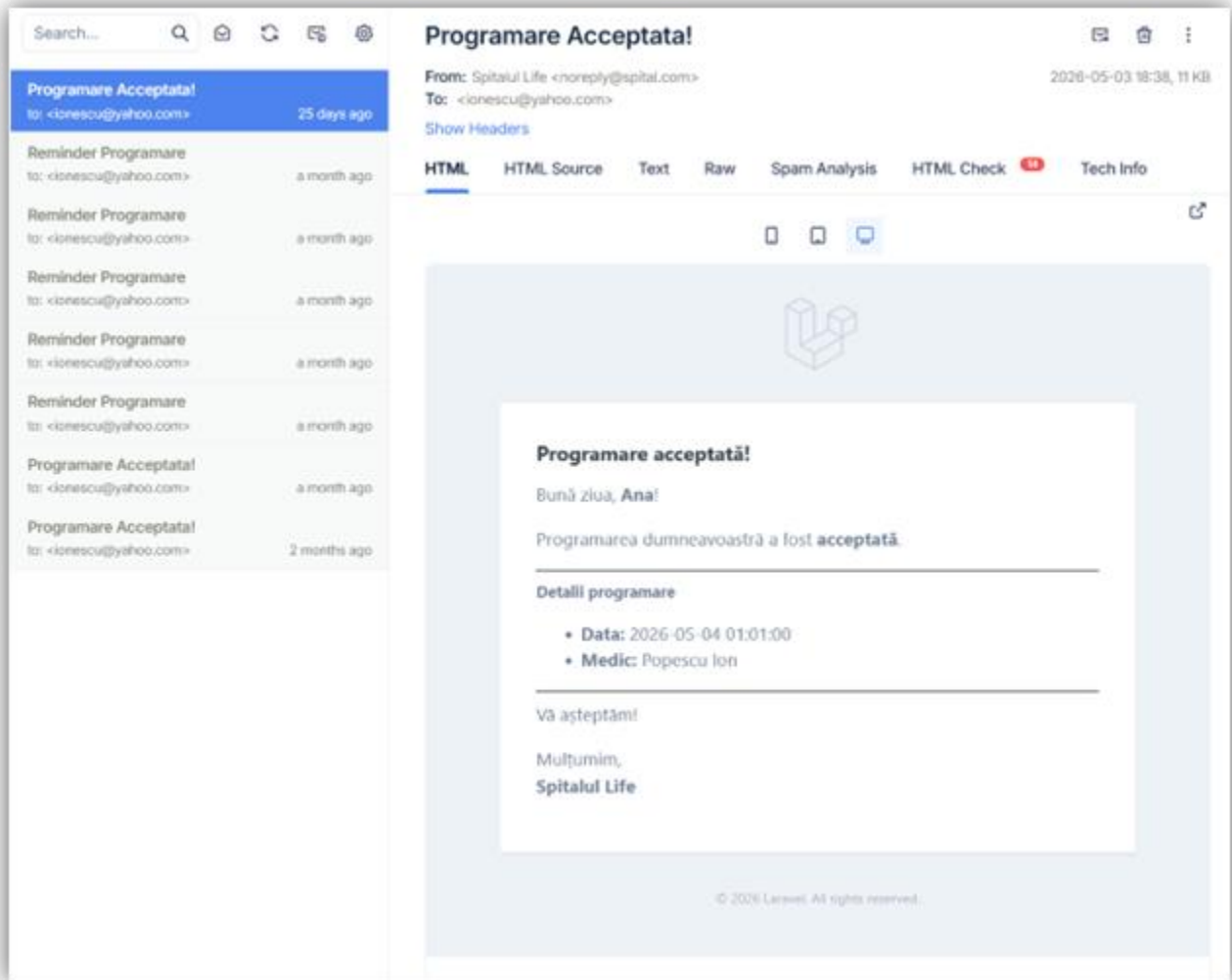
Vă așteptăm!

Mulțumim,

Spitalul Life

```
@endcomponent
```

Această pagină va arăta pentru client astfel:



Figură 18 – Email primit de un pacient în urma confirmării unei programări

Logica din spate se află în `ProgramareAcceptataMail.php`, unde efectiv preluăm pagina blade menționată anterior și o transformă în email:

```
class ProgramareAcceptataMail extends Mailable
{
    public function __construct($programare) {
        $this->programare = $programare;
    }
}
```

```

public function envelope() : Envelope
{
    return new Envelope(
        subject: 'Programare Acceptata!', );
}
public function content() : Content
{
    return new Content(
        markdown: 'mail.programari.acceptate',
        with: [
            'programare' => $this->programare]);
}

```

Trimiterea acestui obiect este gestionată, ulterior, de către `TrimitReminderProgramari.php`:

```

class TrimitReminderProgramari extends Command
{
    protected $signature = 'app:trimit-reminder-programari';
    public function handle()
    {
        $dataMaine = Carbon::tomorrow()->toDateString();
        $programari = Programare::where('data', $dataMaine)->get();
        foreach ($programari as $programare) {
            $user = $programare->pacient->user;
            Mail::to($user->email)->send(
                new ReminderProgramareMail($programare));
        }
        return 0;
    }
}

```

Într-un final, pentru a automatiza trimiterea acestor email-uri, am folosit scheduler-ul framework-ului. De îndată ce executăm `php artisan schedule:work`, se va analiza dacă trebuie trimis un mail, și dacă e cazul, se va trimite. Acest lucru este definit în `console.php`:

```

Schedule::command('app:trimit-reminder-programari')->
    >dailyAt('08:00');
Schedule::command('app:trimit-reminder-retete')->dailyAt('08:00');
Schedule::command('app:trimit-reminder-tratamente')->
    >dailyAt('08:00');

```

```
Schedule::command('app:trimite-reminder-trimiteri')->dailyAt('08:00');
```

Avantaje și Dezavantaje

Un prim avantaj ar fi faptul că aplicația reduce semnificativ volumul de muncă manuală legată de gestionarea medicilor și specializările lor, pacienților, programărilor, rețetelor, tratamentelor și trimerilor, precum și actualizarea fișierelor medicale.

Această aplicație permite accesul rapid la istoricul medical al pacienților, unde medicul are acces instant la programări viitoare și trecute, diagnostice anterioare, tratamente efectuate și rețete eliberate. Acest lucru reduce timpul petrecut cu căutarea informațiilor, crește calitatea deciziilor medicale și îmbunătățește colaborarea între medici.

Un alt avantaj constă în separarea clară a rolurilor utilizatorilor aplicației. Avem 3 tipuri de utilizatori: medicul, care gestionează pacienții și datele medicale, administratorul, care gestionează medicii și specializările lor, și pacientul, care își vizualizează datele personale și poate solicita programări. În acest fel, avem securitate sporită, prevenirea accesului neautorizat la date sensibile, responsabilități clar delimitate.

La nivelul framework-ului, Laravel oferă o arhitectură modernă aplicației. Acest lucru constă în securitate implicită (CSRF, XSS, SQL Injection protection), migrații pentru baza de date, modele Eloquent pentru interacțiuni sigure, validare server-side robustă, structură MVC clară. Astfel, avem un cod modular, ușor de extins, ușor de întreținut și scalabil.

Aplicația are o interfață prietenoasă și intuitivă, folosirea de HTML, CSS și JavaScript permițând lucruri precum formulare intuitive, validare client-side, feedback instant și navigare rapidă între module. În acest mod, un flux de lucru intuitiv reduce timpul de învățare pentru utilizatori.

Astfel, aplicația se aseamănă cu un sistem EMR/EHR ce permite administrarea pacienților, medicilor, gestionarea diagnosticilor, generarea tratamentelor și afișarea și modificarea programărilor. În acest fel, avem o bază pentru sisteme mai mari.

Folosirea framework-ului Laravel, împreună cu o bază de date SQL permite integrare cu API-uri externe (laboratoare, farmaceutice), integrare cu sisteme de notificări prin email/SMS și extindere cu module noi (plăți, rapoarte statistice, arhivare documente).

Printre dezavantaje, se numără limitări în ce privește volume mari de date. Pentru un spital mic sau mediu aplicația funcționează excelent, însă pentru spitale mari, cu mii de pacienți, sute de medici și milioane de înregistrări, soluția ar necesita optimizări de performanță, caching și distribuție pe servere multiple. Laravel este scalabil, dar configurarea necesită expertiză.

Un alt dezavantaj ar consta în lipsa funcțiilor avansate medicale, precum integrare de imagistică medicală (radiografii, RMN), algoritmi de analiză a riscurilor, AI pentru sugestii medicale și schimb de date între spitale. Aceste funcționalități ar crește complexitatea, dar sunt necesare în sistemele mari.

Aplicația în sine, fiind web-based, nu poate fi utilizată offline, deci în zone fără internet (sate, ambulanțe) nu poate fi accesată. O soluție ar fi implementarea unui mod offline sync.

De asemenea, aplicația ar necesita mentenanță periodică din moment ce orice proiect web are nevoie de update-uri Laravel, update-uri pachete Composer, securizarea serverului și backup-uri constante. Fără un administrator tehnic, aplicația ar putea deveni vulnerabilă.

Revenind la validarea client-side menționată, aceasta poate să nu fie suficientă, din moment ce JavaScript poate fi dezactivat, și astfel validarea se poate ocoli. Desigur că avem validarea server-side oferită de Laravel, dar partea client-side rămâne o facilitare, nu o garanție.

Am mai putea avea posibile probleme cu responsabilitatea datelor. Datele medicale sunt extrem de sensibile, deci un sistem real de spital trebuie să respecte GDPR, reguli de anonimizare, criptarea în repaus și în tranzit și audit logging. Aplicația actuală nu implementează complet aceste standarde.

Pentru această aplicație web, ca și optimizări putem să aplicăm în întregime și un framework front-end, pentru a face aplicația să arate mai modernă. Printre astfel de soluții, există Vue.js, React, Livewire, care au ca beneficii interfețe reactive, actualizări în timp real fără reload și UX mult mai profesional.

Putem să implementăm un sistem de permisiuni avansate, adică în loc de doar medic și admin, putem avea și un recepționar, asistent medical, super-admin, medic șef de secție, etc.

Aplicației web îi lipsește un sistem de notificări, care să trimită email pentru confirmări de programări, notificare către medic când un pacient lipsește, remindere automate pentru tratamente, etc. Laravel are sisteme integrate pentru acest lucru.

La nivelul bazei de date, putem să optimizăm cu ajutorul indexărilor, pentru a avea interogări mult mai rapide (ex: index pe email, id-uri..).

Un lucru care ar ajuta aplicația web ar fi un API extern, care ar facilita o aplicație mobilă viitoare sau o integrare cu alte sisteme.

Funcționalități Distinctive

Majoritatea soluțiilor comerciale de tip EMR/EHR sunt extrem de complexe, costisitoare, greu de personalizat și dependente de furnizori externi. Aplicația dezvoltată în cadrul acestui proiect se diferențiază prin simplitate controlată, fiind orientată către spitale mici și medii, clinici private sau cabinete medicale.

Spre deosebire de soluțiile cloud comerciale, datele sunt stocate local, spitalul deține control total asupra informațiilor și nu există dependență de furnizori externi. Acest aspect este esențial în contextul GDPR și al protecției datelor medicale.

Avem o separare clară a responsabilităților medicale și administrative. Soluțiile existente combină adesea roluri multiple într-un singur cont. Aplicația dată însă propune roluri clar

definite (medic / administrator), acces strict doar la informațiile relevante și reducerea riscului de manipulare neautorizată a datelor.

Sistemul dat facilitează de un flux de lucru adaptat realității medicale, fiind construit în jurul activităților reale ale medicului, precum adăugare/editare/ștergere pacient, actualizare fișă medicală, vizualizare istoric, introducere/editare/ștergere de diagnostice, rețete, tratamente, trimiteri, precum și programări. În acest mod, nu există funcții inutile care să îngreuneze utilizarea.

Deși aplicația nu introduce o inovație radicală la nivel global, ea propune inovații relevante prin integrare, adaptare și optimizare tehnică. Proiectul dat utilizează coerent Laravel pentru arhitectura backend, SQL pentru gestionarea relațiilor complexe și JavaScript pentru validare și UX îmbunătățit. O astfel de integrare asigură consistența datelor, securitate sporită și o experiență de utilizare fluentă.

Cu alte cuvinte, au fost introduse tehnici precum validare dublă a datelor (client-side + server-side), model relațional medical bine structurat, și o arhitectură modulară și extensibilă.

Prin validarea dublă a datelor, sistemul implementează validare JavaScript pentru feedback instant și validare Laravel pentru securitate. Ea reduce erorile introduse de utilizatori, previne atacuri de tip injection și îmbunătățește calitatea datelor stocate.

Prin model relațional, baza de date reflectă relațiile reale dintr-un spital. De exemplu, un medic nu poate avea decât o singură specializare, însă o specializare poate avea mai mulți medici, un pacient poate avea mai multe programări, însă o programare nu poate avea decât un singur pacient, etc. Prin acest lucru, obținem o coerență logică a informațiilor, interogări eficiente dar nu în ultimul rând extensibilitate ulterioară.

Prin arhitectură modulară și extensibilă înțelegem ceea ce Laravel ne oferă prin MVC: adăugarea rapidă de module noi, refactorizare fără impact major și separarea clară a logicii de prezentare.

Astfel, Aplicația oferă o soluție web sigură, flexibilă și ușor de utilizat pentru gestiunea istoricului medical, adaptată nevoilor reale ale instituțiilor medicale de dimensiuni mici și medii, punând accent pe controlul datelor, securitate și eficiență operațională.

